Systems-Level Optimizations for Structured Block Propagation in Blockchains

Final Project Proposal (Updated for Kadcast-NG Broadcast)

Arohan Hazarika (22b3948)
Aryan Gupta (22b2255)
Vishrut Nikunj Mittal (22b2427)

Update vs. our initial proposal.

- We will build a **vanilla BTC-like discrete-event simulator first**, then implement **Kadcast-NG broadcast** over the same simulator, and finally explore **systems-level optimizations** (caching + scheduling + parallelism).
- We **do not work on reliability/transport**. We assume a **lossless network (no packet drops)** and optimize the **broadcast structure and implementation pipeline**.

1 Abstract

Broadcast dominates blockchain networking cost: both blocks and transactions must be disseminated quickly and widely. Unstructured gossip is robust but creates substantial redundant traffic and unpredictable propagation delays. We propose to (i) implement a clean, minimal discrete-event simulator for a Bitcoin-like P2P network, (ii) implement Kadcast-NG as a structured broadcast overlay within the simulator, and (iii) treat Kadcast-NG as an implemented system and study performance improvements from caching, pipelining/scheduling, and parallel execution. Our key methodological choice is to assume a lossless network (no drops) and isolate the gains from broadcast structure and systems-level bottlenecks (cache/DRAM stalls, state reuse, contention).

2 Introduction

Motivation

Blockchains require fast propagation of transactions and blocks to maintain security and throughput. In many deployments, dissemination is still based on unstructured overlays, which can flood the network with duplicates. Structured overlays (Kadcast-family) aim to reduce redundant relays by controlling fanout and dissemination paths. However, protocol-level gains can be dominated by implementation effects: serialization/validation costs, cache misses, and scheduling/parallelism decisions. Our project is motivated by the hypothesis that *significant additional gains* are achievable when structured broadcast is optimized as a system.

Problem Statement

We aim to answer:

1. Under identical workloads and network-delay models, how does Kadcast-NG structured broadcast compare to a vanilla BTC-like broadcast baseline in propagation latency and overhead?
2. Which systems-level mechanisms (caching, eviction policies, pipelining/scheduling, parallelism) most improve end-to-end propagation performance?
3. How sensitive are these improvements to topology (dense vs. sparse connected graphs) and to workload stress (arrival intensities for blocks/transactions)?

3 Related Work

Bitcoin popularized peer-to-peer dissemination of blocks and transactions. Kademlia provides a structured overlay based on XOR distance and k-buckets. Kadcast constructs a structured broadcast tree over a Kademlia-style overlay to reduce redundant propagation paths. Kadcast-NG builds on this line of work and refines the broadcast design and evaluation for blockchain settings. In this project, we focus specifically on the **broadcast overlay and implementation pipeline**, not on reliability or adversarial behavior.

4 Proposed Work

We structure the work in three stages to ensure controlled comparisons.

Stage 1: Vanilla BTC-like Discrete-Event Simulator (Baseline)

We will implement a minimal discrete-event simulator (DES) with:

- **Nodes:** $N \approx 100$ peers maintaining an abstract mempool and a blockchain tip.
- **Topology:** random graphs that are either **dense** or **sparse** but **connected**.
- **Lossless network assumption: no packet drops**. All messages delivered after a modeled delay.
- **Workload model:** transaction arrivals and block-generation events modeled as Poisson processes (exponential inter-arrival).
- **Delays:** network delays modeled stochastically (baseline exponential); compute delays modeled as explicit service times (serialization/validation as events).
- **Event types :**
  - Tx arrival, Tx relay send/receive, mempool insert
  - Block creation/arrival, validation start/end, append decision
  - Broadcast send/receive (advertise + payload), link-delay events
  - (Later) cache/DRAM access events

Stage 2: Kadcast-NG Broadcast Overlay in the Simulator

We will implement Kadcast-NG broadcast atop the same DES:

- **Overlay structure:** assign node identifiers and construct the Kadcast/Kademlia-style routing structure.
- **Broadcast algorithm:** structured forwarding responsibilities along subtrees / buckets with tunable fanout.
- **Scope:** broadcast efficiency and latency under lossless delivery (no retransmissions modeled).
- **Outputs:** direct comparison vs. vanilla baseline using identical workloads/topologies/delay models.

Stage 3: Systems-Level Optimizations (Main Contribution)

We will treat Kadcast-NG as a system pipeline and optimize performance by modeling and reducing bottlenecks.

3.1 Cache/DRAM modeled as events

We will model memory access as first-class events:

- Cache hit: small latency
- DRAM access: larger latency
- Parameter sweeps over latencies and cache sizes to study sensitivity

3.2 Three L1 caches (logical separation)

We maintain three logical L1 caches per node:

1. **Block broadcast cache:** recent block data / forwarding metadata used during block propagation.
2. **Transaction cache:** hot transactions and recently forwarded inventories.
3. **Kadcast-NG state cache:** routing/neighbor state and per-broadcast forwarding decisions.

3.3 Eviction policy exploration

We will compare eviction policies under fixed cache sizes:

- LRU, FIFO, Random (baseline set)
- LFU / approximate LFU (time permitting)
- Hybrid schemes (time permitting)

3.4 Parallelization strategy

To handle many replications (graphs/seeds/sweeps), we will exploit multi-core CPUs by:

- running independent simulations in parallel (different seeds/topologies/workloads),
- parallelizing metric computation and plot generation.

5 Evaluation Plan

Metrics

- **Propagation latency:** time-to-reach-x% nodes (50/90/95/99%), plus latency CDFs.
- **Overhead:** messages sent, bytes sent, and duplicate relays.
- **Pipeline cost:** time spent in validation/serialization, cache-hit ratios, DRAM stall time.
- **Stability proxy:** propagation delay relative to block interval (stale-block risk proxy).

Controlled Factors

We vary one factor at a time while keeping others fixed:

- Topology: sparse vs. dense connected graphs (same N).
- Workload: block/txn arrival intensities.
- Network-delay distribution parameters (baseline exponential).
- Kadcast-NG parameters (fanout/redundancy knobs).
- Cache sizes and eviction policies; cache/DRAM latency parameters.

6 Project Timeline

Milestone	Deliverables & Goals
First Progress Report (Approx. Feb 20)	• Vanilla BTC-like DES: event queue, node model, connected topology generator (sparse/dense). • Baseline broadcast implementation + initial metrics (latency, overhead).
Second Progress Report (Approx. March 20)	• Kadcast-NG broadcast overlay implemented in the DES. • Controlled comparisons vs. baseline across multiple graphs/workloads.
Final Report (Near end of semester)	• Systems-level optimizations: cache/DRAM event modeling, 3 L1 caches, eviction policy study. • Full sweeps + plots + conclusions/recommendations; reproducible code release.

7 Resources Required

- **Compute:** multi-core CPU machine (64–128 cores) to run many simulation replications in parallel.
- **Memory:** 32–128 GB RAM recommended for parallel runs and result storage.
- **Optional:** cloud CPU instances for faster sweeps if local capacity is limited.

8 Progress Made So Far

This section summarizes cumulative progress from Checkpoint 1 and Checkpoint 2 as a single continuous narrative.

Work Completed So Far

- **Checkpoint 1 (Vanilla BTC-like DES):** We completed the baseline discrete-event simulator with a global event-priority queue, node-level mempool and blockchain-tree maintenance, random connected topology generation, Poisson transaction generation, PoW mining with exponential inter-arrival times, longest-chain fork resolution, and block propagation/validation with baseline logging support.
- **Checkpoint 2 (Kadcast-NG Integration):** We integrated Kadcast-NG structured broadcast into the same simulator for controlled baseline comparison, using a static connected network of 800 nodes with different spatial densities, routing and forwarding over a Kademlia-style identifier space, and XOR distance as the non-Euclidean metric for forwarding decisions.

Algorithm Details of Kadcast-NG

Kadcast reuses the **Kademlia overlay** as a broadcast substrate: each node is assigned a unique **L-bit identifier** and maintains neighborhood state using **k-buckets**. For a node with local identifier ID_0, bucket B_i stores peers whose identifiers satisfy $2^i \leq d(ID_0, ID_j) < 2^{i+1}$, so nearby identifier regions are tracked in low-index buckets while high-index buckets cover exponentially larger regions. The distance function is the **XOR metric**, $d(x, y) = (x \oplus y)_{10}$, which is **symmetric**, **deterministic**, and **computationally cheap**; this makes routing boundaries stable and enables scalable forwarding decisions without geometric coordinates.

In the **redundant broadcast algorithm**, the sender starts with broadcast height h, chunk c, known-chunk set C, and redundancy factor β. If $c \in C$, the message is aborted immediately to enforce duplicate suppression; otherwise, the chunk is marked known and forwarded level-wise by selecting up to β targets from each bucket B_i for levels $i = 0$ to $h - 1$. This yields structured dissemination with controlled redundancy rather than blind flooding. The parameter β acts as the main **reliability–overhead knob**: higher values increase path diversity and robustness but add traffic, while lower values reduce overhead and duplicate forwarding.

Integration with Simulator Timing Model

First, we model delay using a node-centric upload channel abstraction combined with link-level propagation and transmission effects. Each node maintains $P = 8$ parallel upload channels, sharing the node's total upload bandwidth equally. When a message of size m KB is scheduled for transmission, it is assigned to the least-busy channel. The upload start time and transmission delay are given by:

$$t_{\text{upload}} = \max(t_{\text{request}}, t_{\text{slot_free}}), \qquad t_{\text{tx}} = \frac{m}{c_i^{\text{channel}}},$$

$$t_{\text{arr}} = t_{\text{upload}} + t_{\text{tx}} + d,$$

where t_{request} is the message-ready time, $t_{\text{slot_free}}$ is the next-free time of the assigned channel, $c_i^{\text{channel}} = c_{\text{node}}/P$ is the per-channel bandwidth, and d is the propagation delay along the path. This formulation cleanly separates **queueing (channel wait)**, **transmission**, and **propagation** components.

Second, contention is captured through **finite parallel FIFO queues** at each node rather than packet drops or infinite buffers. Each upload channel operates as an independent FIFO queue, and messages are dynamically assigned to the earliest available channel. Under high load, messages experience increased waiting time before transmission, accurately modeling bandwidth contention and concurrent communication. Thus, congestion manifests as time shifts in channel availability rather than loss, enabling direct measurement of queuing effects.

Third, routing is based on **shortest paths in propagation delay** computed using Dijkstra's algorithm on the underlay graph. For each source–destination pair (s, t), the simulator precomputes:

$$\text{next_hop}(s, t), \quad d(s, t), \quad h(s, t),$$

representing the first hop, total propagation delay, and hop count respectively. This allows efficient routing decisions while keeping the dynamic effects of queueing and transmission delays during execution.

Fourth, the two protocols differ fundamentally in how delay accumulates. In **Classic flooding**, messages are propagated via store-and-forward relays. Each hop incurs its own upload queuing

delay, transmission delay, propagation delay, and an additional protocol-level queuing delay:

$$\Delta^{\text{hop}}_{\text{classic}} = q_{\text{classic}} + \text{upload_wait} + d^{\text{prop}}_{uv} + \frac{m}{\text{effective speed}},$$

and end-to-end delay is the sum across hops. In contrast, **KADcast** uses structured overlay broadcast, where data is sent directly from the source to selected delegates over shortest paths. Upload contention occurs only at the source, while intermediate nodes contribute only lightweight forwarding delays:

$$T_{\text{kadcast}} = t_{\text{upload}}(s) + d(s,t) + \frac{m}{c^{\text{channel}}_s} + \sum q^{\text{kadcast}}_j.$$

This distinction captures the key difference: Classic accumulates queuing and transmission costs at every hop, whereas KADcast centralizes transmission at the source and relies on faster underlay forwarding.

Simulation Setup for Reported Plots

We ran comparative simulations using the same executable and workload settings for both protocols. All runs use identical network topology generation and simulation seeds to ensure fairness and reproducibility. Both Classic flooding and KADcast operate on the same underlying graph, with identical node characteristics, transaction generation processes, and block generation dynamics. This ensures that all observed differences arise solely from the dissemination protocol. The density of the network topology is mainly determined by the degree of the different nodes/vertices in the graph.

Metrics Explained (Network Load and Delay Percentiles)

- **Part 1 – Network load metrics:** **Total messages sent** counts overlay-level transmissions, where each logical send from a source to a destination is counted once, regardless of intermediate forwarding hops. **Total traffic** measures the aggregate data volume carried by the network. These metrics capture dissemination overhead: lower values indicate more efficient use of bandwidth and reduced redundancy.

- **Part 2 – Delay percentile metrics:** For each block b, the propagation delay to node n is defined as

  $$\delta_b(n) = t_{\text{arrival}}(b,n) - t_{\text{creation}}(b).$$

  For each block, percentiles of $\delta_b(n)$ are computed across all nodes, and the reported metrics are aggregated across blocks. We focus on **p50** (median latency), **p90** (upper-tail latency), and higher percentiles to capture worst-case dissemination behavior. These metrics quantify both typical and tail propagation performance.

- **Part 3 – Consensus metric:** **Stale rate** is defined as the fraction of blocks not included in the longest chain:

  $$\text{stale rate} = \frac{\text{stale blocks}}{\text{total blocks}}.$$

  This directly reflects the impact of propagation delay on fork formation and consensus inefficiency.

- **Part 4 – Joint interpretation:** Network load metrics capture **bandwidth efficiency**, delay percentiles capture **latency robustness**, and stale rate captures **consensus stability**. Together, they expose the fundamental trade-off between minimizing communication overhead and ensuring fast, reliable block dissemination across the network.

Experimental Analysis

All 9 runs share identical parameters: 800 nodes, $z_0 = z_1 = 0.5$, txn_iat = 10000, block_iat = 600, duration = 40000, and seeds 49/41.

Classic Baseline

Table 1: Classic Baseline Performance

Metric	Sparse [3,6]	Moderate [10,25]	Dense [50,80]
Messages	529K	2.55M	6.68M
Traffic (MB)	1125	4914	11975
p50 Latency (ms)	1171	983	1129
p90 Latency (ms)	1433	1171	1368
Coverage (%)	95.8	95.9	100
Stale Rate (%)	61.9	50.9	72.6

Classic results are identical across all β values since β only affects KADcast. This consistency validates the correctness of the simulator.

KADcast: Effect of β Across Densities

Latency (p50)

Table 2: KADcast p50 Latency (ms)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	3205	1801	1480
Moderate	1228	715	613
Dense	736	439	336

Latency decreases monotonically with increasing β across all densities. Additionally, increasing density also reduces latency. This indicates that higher redundancy (larger β) and better connectivity both improve propagation speed.

Latency (p90)

Table 3: KADcast p90 Latency (ms)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	4693	2274	1891
Moderate	1789	941	789
Dense	1095	595	438

The p90 latency shows perfectly smooth monotonic behavior, reinforcing that increased β consistently improves worst-case propagation delays.

Coverage

Table 4: KADcast Coverage (%)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	87.8	95.8	98.1
Moderate	98.5	99.8	96.7
Dense	97.0	96.9	100

Coverage generally increases with β, except for a small dip at $\beta = 5$ in moderate density. This deviation is minor and can be attributed to stochastic variation.

Messages

Table 5: KADcast Messages

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	102K	716K	1.18M
Moderate	137K	1.23M	1.89M
Dense	168K	1.42M	1.79M

Messages increase monotonically with β, reflecting the expected increase in redundancy. Higher β improves robustness at the cost of increased message overhead.

Traffic

Table 6: KADcast Traffic (MB)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	191	1392	2191
Moderate	279	2555	4127
Dense	343	3163	3514

Traffic follows the same monotonic trend as messages, confirming that bandwidth usage grows with redundancy.

Stale Rate

Table 7: KADcast Stale Rate (%)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	80.8	66.1	66.1
Moderate	65.8	61.8	50.0
Dense	51.6	41.6	38.7

Stale rate decreases as β increases, demonstrating that faster propagation reduces fork rates. The slight plateau in sparse density between $\beta = 3$ and $\beta = 5$ suggests diminishing returns.

KADcast vs Classic

p50 Improvement

Table 8: p50 Improvement (%)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	-174	-54	-26
Moderate	-25	27	38
Dense	35	61	70

KADcast performance improves monotonically with both β and density. It underperforms in sparse settings but significantly outperforms Classic in dense networks.

Message Reduction

Table 9: Message Reduction (%)

	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	81	-35	-122
Moderate	95	52	26
Dense	98	79	73

$\beta = 1$ consistently minimizes message overhead, while higher β trades efficiency for improved latency and coverage. Dense networks strongly favor KADcast due to the high cost of flooding in Classic.

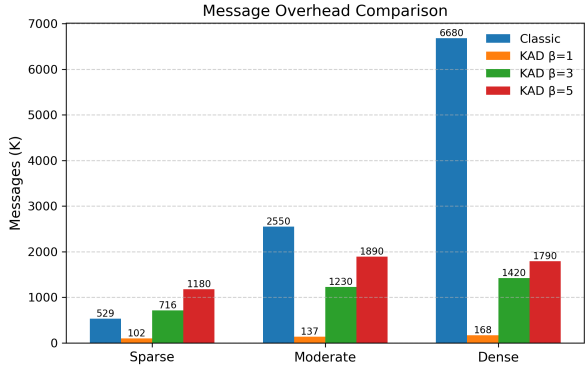
Traffic Reduction

Table 10: Traffic Reduction (%)

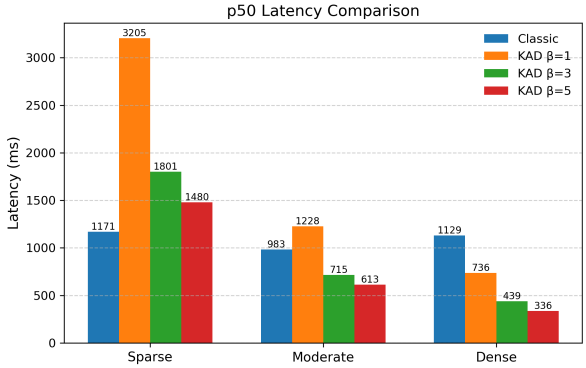
	$\beta = 1$	$\beta = 3$	$\beta = 5$
Sparse	83	-24	-95
Moderate	94	48	16
Dense	97	74	71

Traffic trends mirror message behavior, reinforcing the trade-off between efficiency and propagation speed.

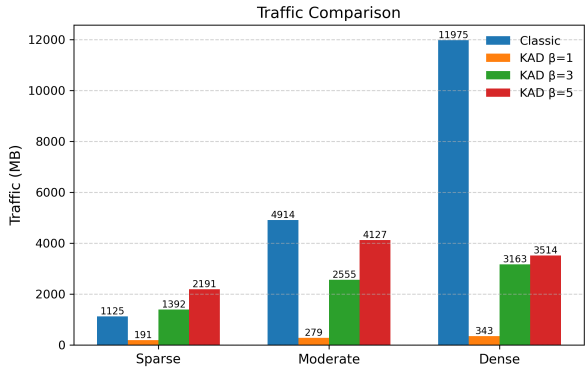
Classic vs KADcast: Visual Comparison



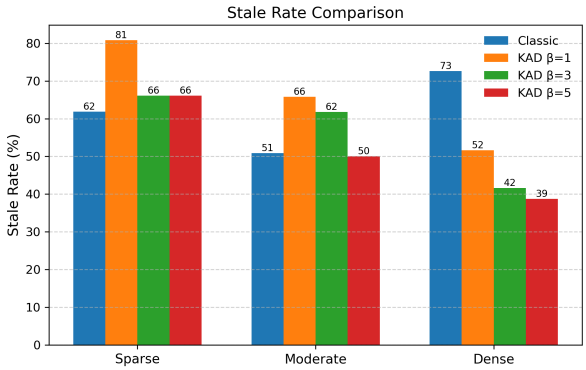
(a) Message Overhead



(b) p50 Latency



(c) Traffic



(d) Stale Rate

Figure 1: Classic vs KADcast comparison across densities and β values

- These bar plots provide a direct visual comparison between Classic flooding and KADcast across key metrics.
- Latency plots clearly show that KADcast outperforms Classic in moderate and dense networks for $\beta \geq 3$, while Classic remains competitive in sparse settings.
- Message and traffic plots highlight the trade-off introduced by β: lower β significantly reduces overhead compared to Classic, while higher β increases redundancy but improves performance.
- Stale rate plots reinforce that faster propagation in KADcast reduces forks, especially in dense networks.

Summary and Insights

All metrics exhibit smooth and predictable trends across both β and network density, forming a consistent 3×3 performance grid.

- β controls the speed–overhead trade-off. $\beta = 1$ is highly efficient (up to 98% traffic savings) but suffers from poor latency and coverage. $\beta = 5$ provides fast propagation and high reliability at the cost of increased bandwidth usage. $\beta = 3$ represents a balanced configuration.

- Network density determines where KADcast outperforms Classic. In dense networks, KADcast achieves up to 70% lower latency due to avoiding flooding overhead. In sparse networks, Classic performs better since flooding remains inexpensive.
- The crossover point occurs at moderate density. For $\beta \geq 3$, KADcast outperforms Classic in moderate and dense regimes, while $\beta = 1$ only outperforms Classic in dense networks.

Novelty and Final Tasks Before Project Completion

- **Forwarding-logic refinement (main novelty):** we plan to modify Kadcast so that **intermediate nodes receive and store block/transaction information**, but **only designated target nodes execute Kadcast forwarding**. This should improve awareness without reintroducing full flooding.
- **Duplicate-target suppression:** if a node receives the same block again as a target, it will not forward redundantly. We expect this to reduce unnecessary relays while preserving useful path diversity.
- **Distance-function and topology study:** we will test an alternative routing distance based on **Jaccard similarity** and evaluate which **graph topologies** (sparse/dense/other structural variants) yield the largest Kadcast gains.
- **Delay-model extension (time permitting):** we plan to enrich the link-delay model with a lightweight approximation for retransmission effects. Because this direction appears more impactful for Kadcast behavior, **cache optimization from the original checkpoint-3 plan is deprioritized** for now.

Code and Reproducibility

The complete implementation (simulator, Kadcast integration, experiment scripts, and plotting pipeline) is available at:
https://github.com/ArohanH/KadSim

References

[1] Petar Maymounkov and David Mazières.
Kademlia: A Peer-to-Peer Information System Based on the XOR Metric.
IPTPS, 2002.

[2] Satoshi Nakamoto.
Bitcoin: A Peer-to-Peer Electronic Cash System.
2008.

[3] Elias Rohrer and Florian Tschorsch.
Kadcast: A Structured Approach to Broadcast in Blockchain Networks.
AFT, 2019.

[4] Elias Rohrer and Florian Tschorsch.
Kadcast-NG: A Structured Broadcast Protocol for Blockchain Networks.
IACR ePrint 2021/996, 2021.
https://eprint.iacr.org/2021/996